App Specification: CropDoc 🌿

AI-Powered Crop Health Monitor, Yield Predictor, and Treatment Advisor

1. Project Overview

- **Hackathon:** Aharada Kaushal Kumbh 2026
- **Target Track:** Smart Agriculture
- **Problem Statement Addressed:** "Leverage IoT and ML to monitor crop health, predict yields, and optimize resource usage."
- **Developer:** Solo Submission

The Pitch

CropDoc is a mobile-first web application designed to put expert agronomist knowledge directly into the hands of farmers. By simply uploading a photo of a diseased plant leaf, the app utilizes a lightweight Machine Learning model to identify the specific pathogen. It then combines this diagnosis with real-time, hyper-local weather data to query a Generative AI LLM. The result is a localized, weather-aware treatment plan, an estimation of potential yield impact, and resource optimization strategies—all available in local vernacular languages.

2. Core Features

- **Instant Disease Identification:** Fast, edge-friendly image classification using transfer learning (MobileNetV2).
- **Resource Optimization (Weather-Aware AI):** Integrates with the Open-Meteo API to check local weather forecasts. The AI uses this to optimize resources (e.g., advising against spraying chemicals if rain is imminent, saving the farmer money and protecting the environment).
- **Yield Impact Estimator:** Calculates potential crop loss based on the identified disease severity and the user's inputted farm size, directly fulfilling the "predict yields" hackathon requirement.
- **Vernacular / Multi-language UI:** A simple toggle to switch the AI's output between English and local languages (e.g., Hindi) to ensure real-world accessibility for Indian farmers.
- **Mobile-First Interface:** A responsive, intuitive UI that works flawlessly on field mobile devices with low bandwidth.

3. Technology Stack (Optimized for Solo Hackathon Build)

Frontend (User Interface)

- **Framework:** HTML + Vanilla JS
- **Styling:** Tailwind CSS (Rapid, responsive, modern UI)

- **Hosting:** Vercel (CI/CD integration directly from GitHub)

Backend (API & ML Serving)

- **Framework:** FastAPI (Python) - *Chosen for native asynchronous support and auto-generating Swagger documentation.*
- **Hosting:** Render (Free tier, Web Service) or Railway.

Machine Learning, AI & External APIs

- **Classification Model:** MobileNetV2 (via TensorFlow/Keras). Pre-trained on ImageNet, fine-tuned on the New Plant Diseases Dataset.
- **Generative AI:** Google Gemini API (Used for generating natural language treatment plans, yield estimations, and translations).
- **Weather Data:** Open-Meteo API (Free, no-key-required API for fetching hyper-local 7-day forecasts based on device GPS).

Data Source

- **Training Data:** "New Plant Diseases Dataset" (Kaggle - `vipoooool/new-plant-diseases-dataset`). Contains 87K+ pre-augmented RGB images of healthy and diseased crop leaves categorized into 38 different classes.

4. System Architecture & Data Flow

1. **Client Interaction:** User captures an image of a leaf, inputs their farm size (in acres/hectares), and allows browser GPS location access on the React frontend.
2. **Data Transmission:** Image, GPS coordinates, and farm size are sent as a `multipart/form-data` payload via a `POST` request to the FastAPI backend.
3. **Parallel Processing:** * **ML Inference:** Python backend resizes the image to `224x224` and the `.h5` model predicts the disease class.
   - **Weather Fetch:** Backend pings Open-Meteo API with the GPS coordinates to get the next 48 hours of precipitation and temperature data.
4. **GenAI Enrichment:** The predicted disease, weather data, and farm size are injected into a dynamic Gemini prompt: *"A farmer's {farm_size} acre crop has {predicted_class}. The local weather for the next 48 hours is {weather_summary}. 1. Provide a 3-step organic treatment plan taking the weather into account (e.g., don't spray if raining). 2. Estimate the potential yield loss percentage if untreated."*
5. **Response Formulation:** FastAPI bundles the prediction, weather context, yield estimate, and treatment plan into a JSON object.
6. **Client Display:** React renders the comprehensive dashboard to the farmer.

5. API Interface Specification

Endpoint: `/api/predict`

- **Method:** `POST`
- **Content-Type:** `multipart/form-data`
- **Payload:** `file` (Image), `latitude` (Float), `longitude` (Float), `farm_size` (Float), `language` (String: "en" or "hi")

Success Response (200 OK):

```
{
  "status": "success",
  "data": {
    "crop": "Tomato",
    "disease": "Early Blight",
    "confidence_score": 0.965,
    "weather_context": "Heavy rain expected in 4 hours.",
    "yield_impact_estimate": "Potential 20-30% yield loss if left untreated.",
    "treatment_plan": "1. Remove and destroy lower infected leaves immediately.\n
  }
}
```

Error Response (400 Bad Request):

```
{
  "status": "error",
  "message": "Invalid file type. Please upload a valid image."
}
```

6. User Experience (UX) Screen Flow

Screen 1: The Landing / Upload Page

- **Header:** Clean "CropDoc" branding with a Language Toggle (EN | HI).
- **Hero Section:** "Detect Diseases. Protect Yields. Optimize Resources."
- **Inputs:** Optional "Farm Size (Acres)" input field.
- **Primary Action:** A massive, easily tappable target area that says "Tap to Take Photo".

Screen 2: The Loading State

- **Visual:** A pulsing scanning animation over the image preview.
- **Text Cycle:** "Analyzing leaf patterns..." -> "Checking local weather forecasts..." -> "Consulting AI agronomist..."

Screen 3: The Results Dashboard

- **Diagnosis Card:** Highlights the identified disease in bold, color-coded text with the confidence percentage.
- **Yield Impact Banner:** A prominent warning banner showing the estimated yield loss if untreated.
- **Smart Treatment Card:** Displays the Gemini-generated 3-step action plan. *Crucially, this section highlights the weather-aware advice (e.g., highlighting "Wait until tomorrow to spray" in yellow).*
- **Action:** "Scan Another Plant" button.

7. Development Milestones (Hackathon Timeline)

- **Phase 1: ML Pipeline (Hours 1-4)**
  - Download "New Plant Diseases Dataset" from Kaggle.
  - Setup Google Colab, load MobileNetV2, freeze base layers, add dense classification layer.
  - Train, evaluate, and export the model (`model.h5`).
- **Phase 2: The Brains/Backend (Hours 5-9)**
  - Initialize FastAPI app.
  - Build the `/predict` route.
  - Integrate the Open-Meteo API for weather fetching.
  - Integrate the `google-generativeai` python SDK to chain the ML output and weather data.
- **Phase 3: The Face/Frontend (Hours 10-14)**
  - Setup Vite + React + Tailwind.
  - Build UI components (including GPS permission request and Farm Size input).
  - Connect frontend to backend using `axios`.
- **Phase 4: Polish & Deploy (Hours 15-18)**
  - Deploy Frontend to Vercel.
  - Deploy Backend to Render.
  - *Crucial:* Prepare live demo dataset and test the location/weather logic.